Deep Computer Vision from Pixels

Executed Technical Project Report

A fully reproducible, from-scratch PyTorch study of geometric shape classification, manual image filtering, intermediate CNN activations, pooling, and robustness.

Dataset	2,100 locally generated grayscale images
Task	Circle / square / triangle classification
Primary experiment	10 epochs, Adam, learning rate 0.001
Held-out evaluation	462 test images, 100.00% accuracy
Deliverables	Python code, weights, raw metrics, eight figures, tests

Report version: October 2026. All numeric findings below come from the executed run stored in `results/metrics.json`, not fabricated or illustrative training curves.

1. Abstract and aims

This experiment traces a computer-vision pipeline from numerical pixels through handcrafted edge filters to a small learned convolutional classifier. The task is purposely narrow: discriminate between circles, squares, and triangles on 32 by 32 grayscale canvases. A complete local generator removes external dataset dependencies and allows straightforward replication.

The main CNN correctly classified all 462 held-out samples in the executed run. A separate no-pooling model required many more parameters and reached 98.27% after five epochs. Additional noise experiments revealed rapid performance deterioration under stronger corruption. Thus perfect accuracy here describes this controlled distribution, not reliable behavior on real photographs.

2. Dataset construction and experimental protocol

For each of three labels, the generator creates 700 images. Shapes vary in center location, size and brightness. The drawings use 4x supersampling followed by Lanczos downsampling; a Gaussian random field is added and values are clipped to [0, 1]. Each image is represented as a float32 tensor with dimensions (1, 32, 32).

The dataset is stratified with 1,638 training examples (78%) and 462 held-out examples (22%): exactly 154 test examples of each class. The Python, NumPy and PyTorch seeds are set to 42. The model trains from random initialization; no pretrained models, third-party datasets, or remote services are used.

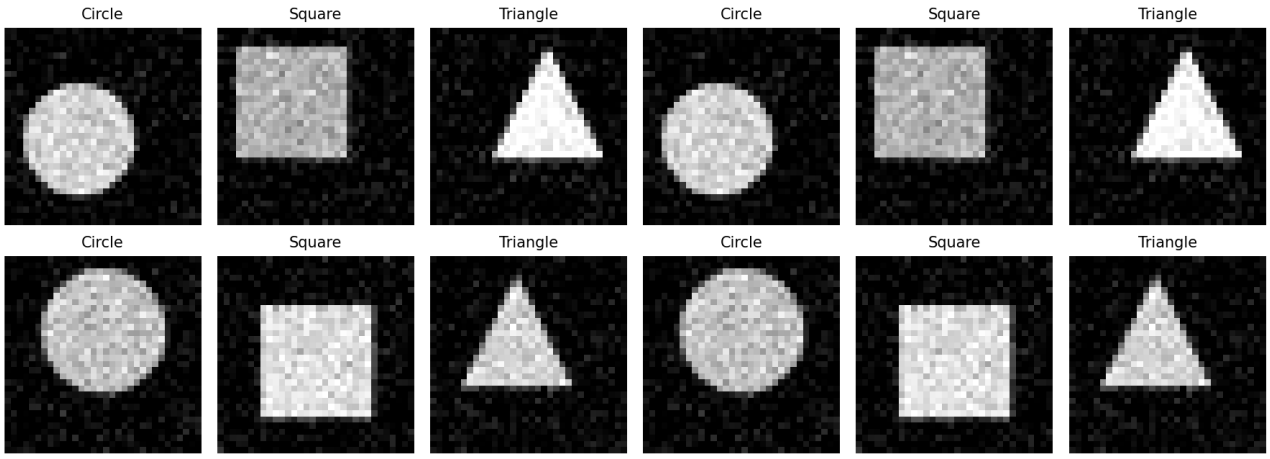


Figure 1. Examples actually generated by the executable pipeline.

3. Manual low-level image processing

The project implements 3 by 3 Sobel kernels by explicit sliding-window multiplication and summation. The horizontal-response kernel has rows [-1, 0, 1], [-2, 0, 2], [-1, 0, 1]; the vertical kernel is its transpose. Magnitude is calculated as sqrt(Gx squared + Gy squared). The operator illustrates how local contrast transitions become edge maps before any CNN weights are learned.

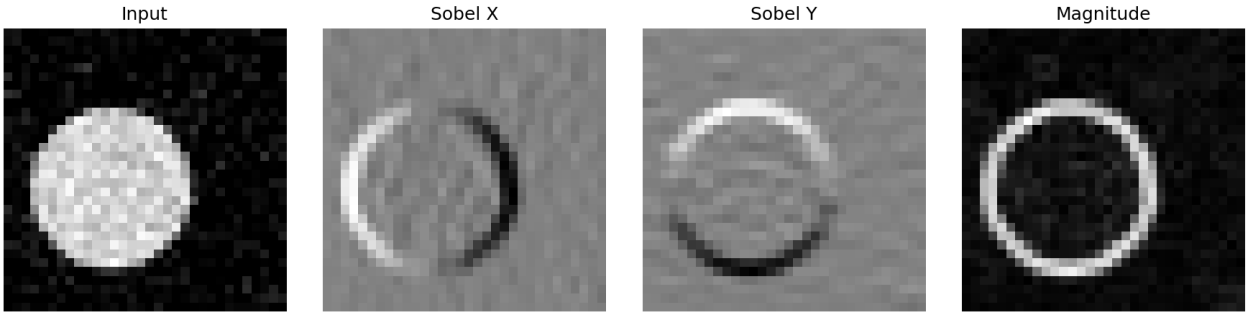


Figure 2. Input pixels, two directional Sobel outputs and gradient magnitude.

4. CNN design and dimensional reasoning

The primary architecture is Conv2d(1,8,3,padding=1), ReLU, MaxPool2d(2), Conv2d(8,16,3,padding=1), ReLU, MaxPool2d(2), Flatten, Linear(1024,32), ReLU, Linear(32,3). The tensor path is (B,1,32,32) to (B,8,16,16) to (B,16,8,8) to (B,1024) to (B,32) to (B,3).

Parameter counts from instantiated PyTorch models are 80 for the first convolution, 1,168 for the second convolution, 32,800 for the 1024-to-32 dense layer, and 99 for the 32-to-3 layer, totaling **34,147 trainable parameters**. ReLU supplies nonlinearity; max pooling halves spatial dimensions twice, reducing the dense input dimension.

5. Training and evaluation

Both networks use Adam with learning rate 0.001 and cross-entropy on raw three-class logits, batch size 64, and deterministic CPU operations. Each batch performs zero_grad, forward pass, loss computation, backward pass and optimizer step. The primary model trains for ten epochs. The code records loss, train accuracy and test accuracy after each epoch.

The test set is evaluated during training for educational visualization only. No best-checkpoint selection or hyperparameter search uses test accuracy; the final epoch-10 weights are always saved. A separate validation split would be a better design for model selection in a larger study.

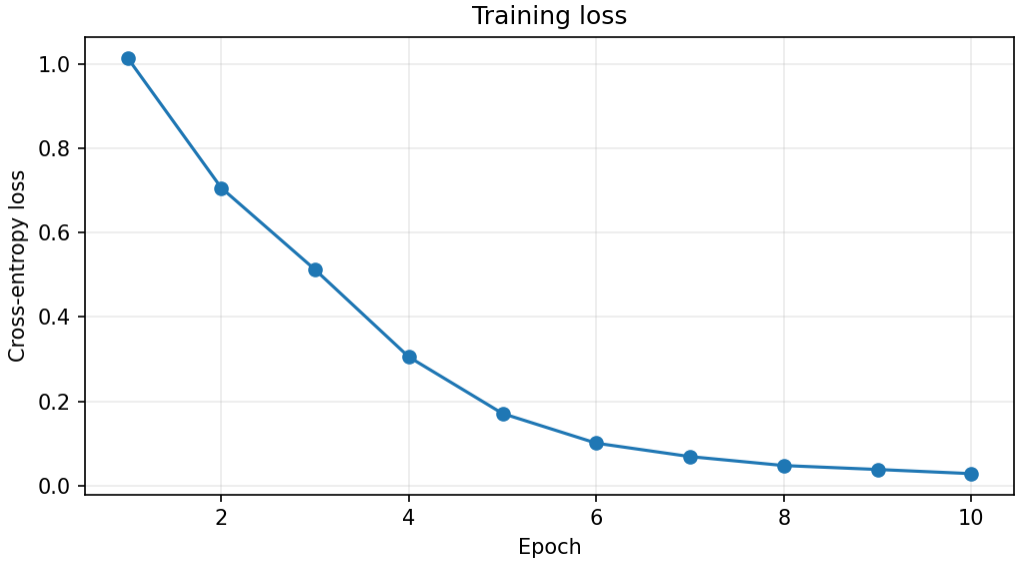


Figure 3. Measured training cross-entropy across ten epochs.

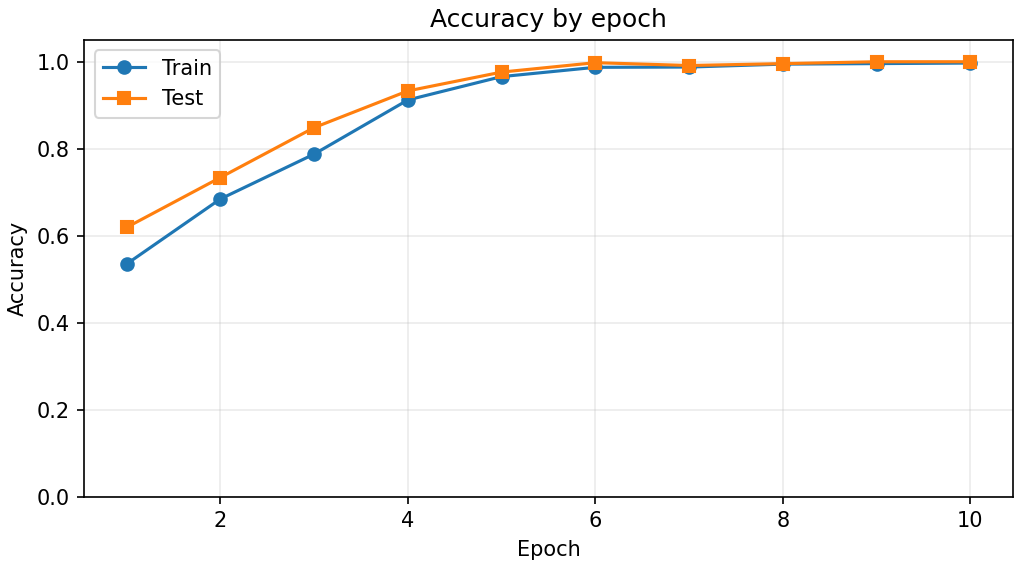


Figure 4. Measured train and held-out test accuracy by epoch.

6. Primary results

Measurement	Observed value
Held-out test accuracy	100.00% (462 / 462)
Circle / square / triangle correct	154 / 154 / 154
Confusion matrix	[[154, 0, 0], [0, 154, 0], [0, 0, 154]]
Final train accuracy	99.69%
Final train loss	0.0287
Primary model parameters	34,147

Precision, recall and F1 are 1.00 for each class on this particular test partition. The result is plausible because these shape categories have strong geometric differences and the test images come from the same generator family as the training images.

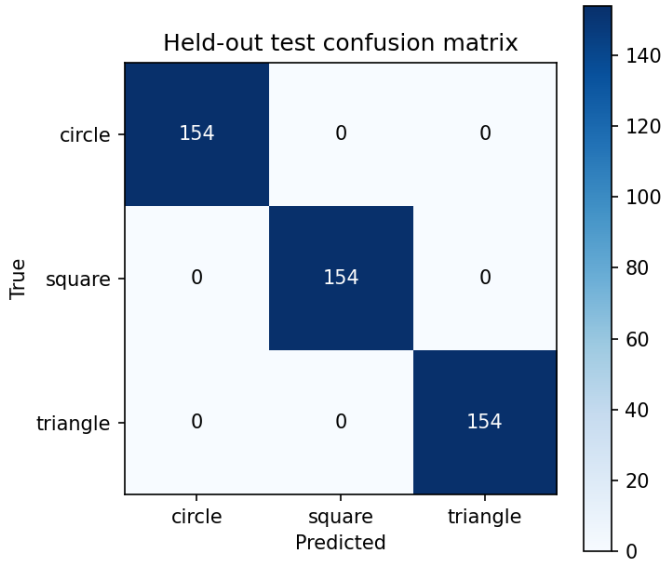


Figure 5. Measured confusion matrix: all 462 held-out samples correctly classified.

7. Feature representation inspection

The first learned convolution transforms the single-channel image into eight activation maps. Visualizing these maps reveals different spatial responses induced by trained filters. They are learned weights, not the manual Sobel kernels, and should not be labeled as explicit semantic shape detectors without additional analysis.

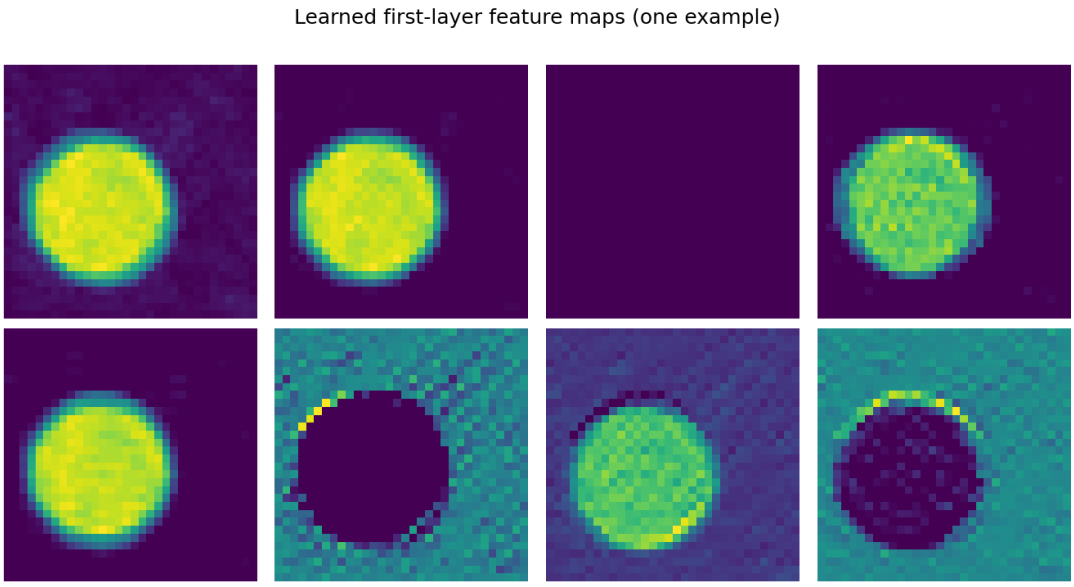


Figure 6. Eight actual first-block activation maps of one sample.

8. Pooling ablation and parameter accounting

A second model substitutes identities for both max-pooling operations. Its feature map therefore remains 16 by 32 by 32 before flattening, versus 16 by 8 by 8 for the pooled network. Flattened dimensionality rises from 1,024 to 16,384 (16x). The no-pooling dense layer contains 16,384 x 32 + 32 = 524,320 parameters, giving **525,667 total** parameters, compared with 34,147 for the pooled network.

The independently trained no-pooling variant achieved **98.27%** after 5 epochs. Because it received only five epochs rather than the main model's ten, this is a model-size experiment and an exploratory performance measurement, not a rigorously controlled comparison of predictive quality. The original illustrative 49,760-parameter figure has been replaced by the exact implemented count.

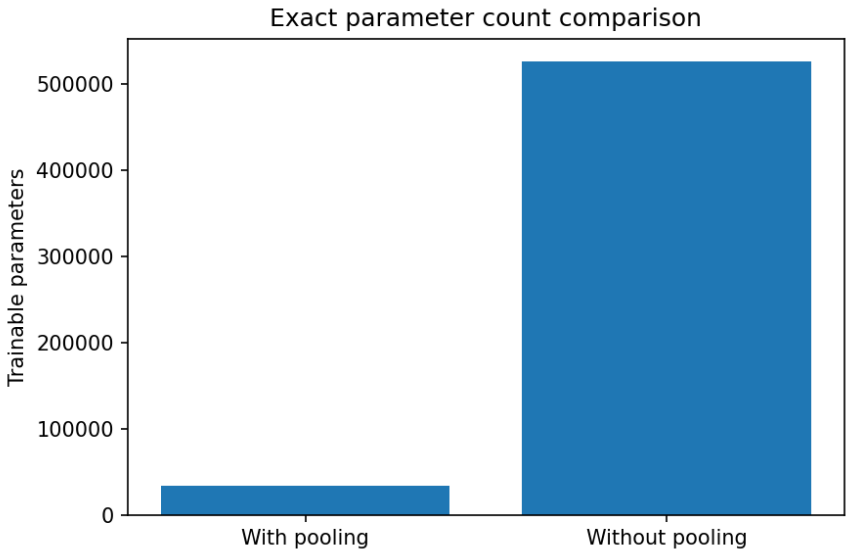


Figure 7. Exact PyTorch trainable-parameter counts.

9. Robustness study

The saved model was evaluated without retraining on fresh, independently generated images: 100 examples per class at each Gaussian noise standard deviation. The test geometry follows the same generator, but noise intensity changes. The results below are empirical, not hypothetical.

Noise standard deviation	Correct / total	Accuracy
0.000	299 / 300	99.67%
0.075	299 / 300	99.67%
0.150	276 / 300	92.00%
0.250	206 / 300	68.67%
0.400	185 / 300	61.67%

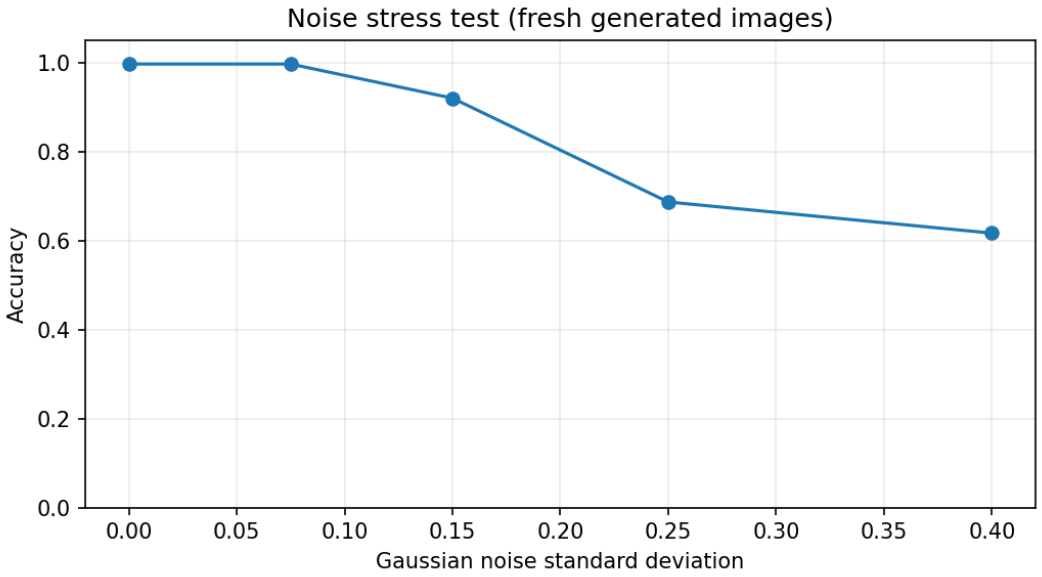


Figure 8. Measured drop in accuracy as Gaussian noise increases.

At sigma 0.25, accuracy fell to 68.67%, and at sigma 0.40 it fell to 61.67%. This is direct evidence of limited noise robustness, even while clean same-distribution classification is near perfect. These measurements do not cover rotation, occlusion, background clutter, or changes in shape distribution.

10. Computational footprint and reproducibility

The primary model finished ten epochs in 4.131 seconds and the no-pooling model finished five epochs in 2.702 seconds in the recorded CPU runtime. Timing is environment-dependent and is not a normalized performance benchmark. The executed environment reported PyTorch 2.10.0+cpu, NumPy 2.3.5, and CPU execution.

Reproduce from the repository root by running: `python -m pip install -r requirements.txt` followed by `python project.py`. The script writes the model checkpoint, full metrics JSON, and all eight plots. Run `python -m unittest discover -s tests -v` to check image reproducibility, expected tensor shapes, parameter counts and Sobel behavior.

11. Limitations and careful interpretation

The classes are visually simple and generated from a single procedural family. Train and test examples are disjoint, but they share generator settings and thus cannot support claims about natural images. The sample size is small relative to modern vision benchmarks. No comparison with ResNet or vision transformers was attempted. The training script does not implement cross-validation, statistical uncertainty intervals or an external dataset benchmark.

Monitoring test accuracy during training is convenient for explanation but not ideal for methodology; the retained weights are always those of the final epoch. No validation split is used, and further tuning against the same test partition would weaken its independence. Robustness results are restricted to added Gaussian noise.

12. Findings and conclusion

A compact convolutional model can learn an accurately separable controlled geometric task from random initialization. Manual Sobel operations clarify low-level edge extraction while learned intermediate feature maps illustrate representation learning. Pooling reduces the flatten dimension by 16x and changes the parameter budget from 525,667 to 34,147 in the compared architectures. Strong noise causes pronounced accuracy deterioration, reinforcing the distinction between perfect in-distribution performance and real-world robustness.

Overall, the project succeeds as a self-contained, executed teaching experiment. Its primary value is the complete inspectable code-to-results pipeline, not the headline accuracy alone.

Appendix A. File provenance and artifact index

Source implementation: project.py. Dependencies: requirements.txt. Test suite: tests/test_project.py. Captured terminal output: results/run.log. Full per-epoch logs, confusion matrix, classification metrics, robustness tests, run configuration and environment: results/metrics.json. Trained checkpoint: results/model.pt. Eight plots: results/figures/. GitHub usage: README.md.

Appendix B. Important equations

Discrete correlation (as implemented for Sobel): Z(i,j) = sum over u,v of K(u,v) X(i+u,j+v). In many deep-learning frameworks the operation called convolution is cross-correlation, with the kernel not flipped.

Rectified linear unit: ReLU(z) = max(0,z). 2x2 max pooling: P(i,j) = max X(2i+u,2j+v) over u,v in {0,1}. Edge magnitude: G = sqrt(Gx^2 + Gy^2). Cross-entropy: L = -mean(log softmax(logits)[true class]). Predicted label: argmax of three logits.